

Assignment 4: Federated Learning for Remaining Useful Life (RUL) Prediction

(Your Name)

January 7, 2026

1 Overview

This assignment implements and compares **centralized** and **federated** training for Remaining Useful Life (RUL) prediction using the NASA CMAPSS turbofan engine degradation dataset (FD001–FD004). The core requirement is a fair comparison between:

- a centralized model trained on all available training data, and
- a federated setup where multiple clients train locally and a server aggregates updates (FedAvg).

2 Project Structure

We designed the codebase as a set of reusable modules plus lightweight orchestrator scripts. This ensured centralized and federated experiments reuse the same model, preprocessing, metrics, and plotting logic (fair comparison and easier debugging).

Modules

- `data_cmapss.py`: Loads the raw CMAPSS text files into DataFrames with correct columns and types.
- `data_windows.py`: Labels RUL, applies preprocessing, builds windowed datasets and PyTorch DataLoaders. Also contains federated partitioning (10 clients $\times$ $\sim$ 10 engines each).
- `model_lstm.py`: LSTM regressor definition ($[B, T, F] \rightarrow [B]$).
- `training.py`: Modular training loop with metrics logging per epoch; evaluation runs under `no_grad()`.
- `plotting.py`: Unified plotting utilities for histories and prediction diagnostics.

Orchestrators

- Centralized runs: `run_centralized*.py`
- Federated runs (Flower FedAvg): `run_federated.py`
- Comparison run: `run_compare_central_vs_fed.py`

3 Dataset and Preprocessing

3.1 Raw Data Organization

CMAPSS is provided as whitespace-separated `.txt` files:

- `train_FD00x.txt`: run-to-failure trajectories (training)
- `test_FD00x.txt`: truncated trajectories ending before failure (testing)

- `RUL_FD00x.txt`: one value per test engine (RUL at the last observed cycle)

Each row corresponds to one cycle snapshot with columns: `unit`, `cycle`, 3 operating settings (`os1--os3`) and 21 sensors (`s1--s21`).

3.2 RUL Labeling

For training trajectories (run-to-failure), we label each row:

$$\text{RUL}(t) = \max(\text{cycle}_{\text{unit}}) - \text{cycle}(t).$$

For test trajectories, the provided `RUL_FD00x.txt` gives RUL_{last} at the final observed cycle. We label each row:

$$\text{RUL}(t) = \text{RUL}_{\text{last}}(\text{unit}) + (\text{cycle}_{\text{last}}(\text{unit}) - \text{cycle}(t)).$$

3.3 Window Generation for the LSTM

We transform trajectories into supervised samples via sliding windows:

- Input $X \in R^{\text{seq_len} \times F}$: last `seq_len` cycles of features (settings+sensors)
- Target y : RUL at the *end* of the window

For test evaluation we use **one window per engine** (the last available window), matching the CMAPSS objective.

3.4 Key Preprocessing Choices

Early training runs produced nearly constant predictions (collapse to mean RUL). The following preprocessing steps were essential:

1. **RUL capping**: cap RUL at 125 cycles (piecewise target). This reduces label imbalance (very large RUL dominates MSE otherwise).
2. **Z-score normalization**: compute mean/std on training data only and apply to train/val/test. This is critical because sensors have different scales.
3. **Split by engine ID**: validation is performed by holding out entire engines to avoid window leakage.

4 Data Inspection and Visualization

We implemented an inspection script to:

- report number of engines and trajectory lengths (rows per engine),
- check missing values / infinities,
- generate required plots:
 1. **Sensor trends per engine**: top varying sensors vs cycle for representative engines.
 2. **RUL vs cycles**: monotonic decreasing trend from high RUL to 0 for training engines.

5 Centralized Training

5.1 Training Setup

We trained an LSTM regressor centrally using the same preprocessing and windowing pipeline. The training loop logs train/val/test metrics per epoch; **test metrics are evaluation-only and do not affect weights**.

Through iterative tuning, we found a stable configuration:

- `seq_len` = 50, `batch_size` = 128, `val_ratio` = 0.1
- LSTM: `hidden_size`=64, `num_layers`=2, `dropout`=0.1

Sequence length had a large impact: longer windows captured degradation dynamics better and improved generalization.

5.2 Centralized Result Interpretation (Figures 1–2)

Figure 1 (Predicted vs True RUL). Points generally follow the diagonal, indicating the model learns a meaningful mapping from sensor history to RUL. A concentration near the upper range is expected because RUL is capped (many targets saturate at 125).

Figure 2 (RMSE over epochs). Train RMSE decreases steadily, while validation and test improve rapidly in early epochs and then plateau. This indicates the model quickly learns the dominant structure and later epochs mostly refine the training fit without improving generalization. This makes checkpointing by `val_rmse` important (best validation epoch typically occurs early).

6 Federated Learning (Flower FedAvg)

6.1 Federated Setup

We implemented server-based federated learning using Flower and FedAvg:

- **Clients:** 10 clients, each holding data from ~ 10 engines (non-overlapping partitions).
- **Local training:** each client trains locally for `local_epochs` per round.
- **Aggregation:** server performs FedAvg (weighted by number of samples).
- **Evaluation:** global model is evaluated on centralized val/test loaders (no gradients).

Preprocessing statistics (feature selection, normalization) are fit once on the global training set and shared across clients, ensuring stable and comparable features.

6.2 Why Federated Curves Differ from Centralized

Federated optimization is harder because:

- **Non-IID data:** clients see different subsets of engines (distribution shift across clients).
- **Client drift:** local updates move in different directions before averaging.
- **Compute mismatch:** one federated “round” is not equivalent to one centralized epoch; the server sees aggregated progress from partial/local steps.

6.3 Comparison Plot Interpretation (Figure 3)

Figure 3 overlays centralized and federated train/val/test RMSE. Centralized training improves quickly because each epoch sees the full distribution. Federated training typically starts worse and improves more gradually across rounds due to the non-IID partitioning and aggregation dynamics. Despite this, federated performance can approach centralized performance with enough rounds and appropriate local training settings.

7 Figures

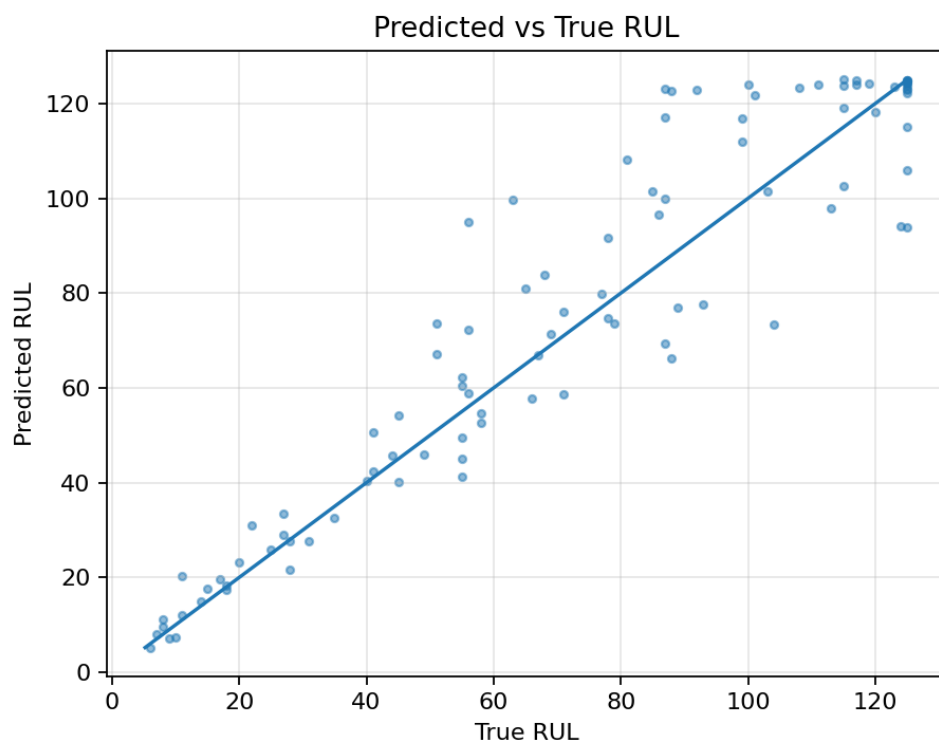


Figure 1: Centralized: Predicted vs True RUL (test engines).

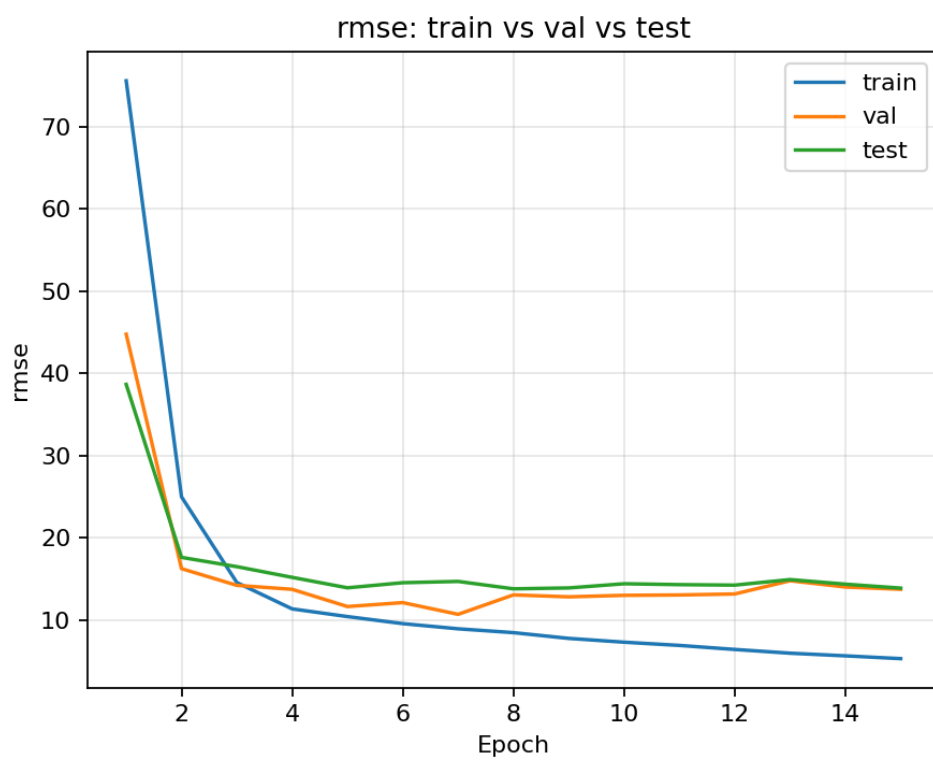


Figure 2: Centralized: RMSE over epochs (train/val/test).

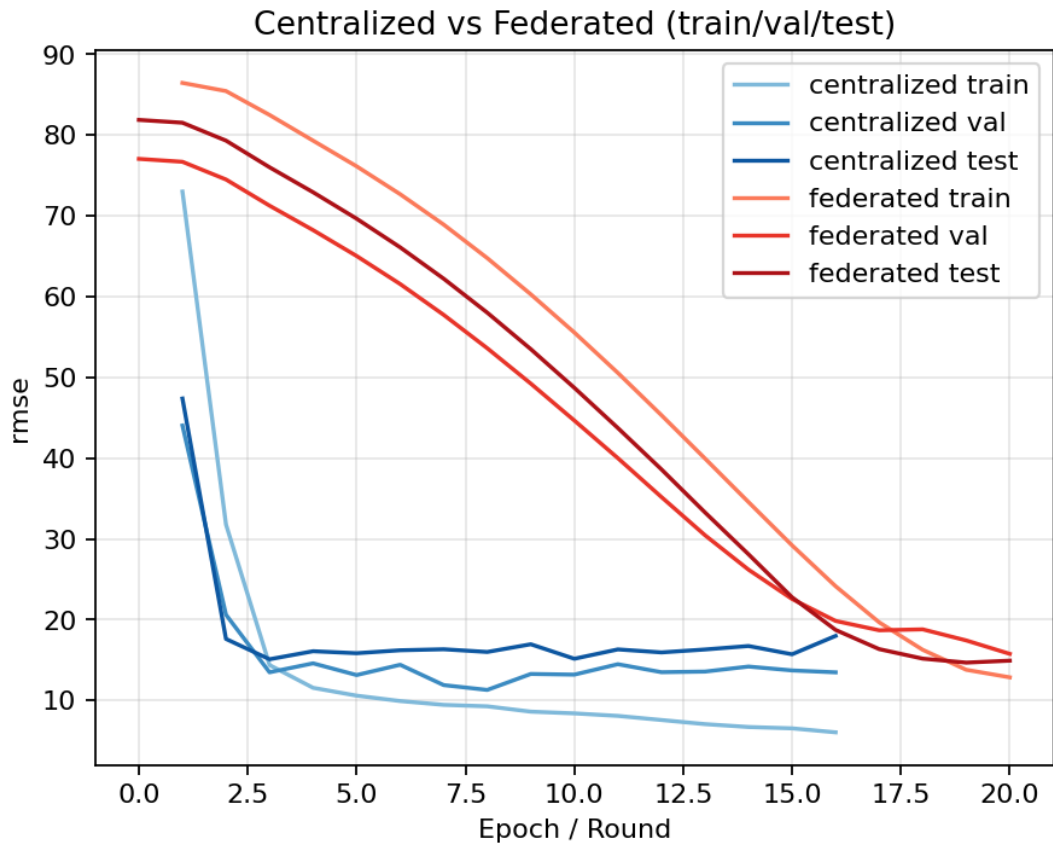


Figure 3: Centralized vs Federated: train/val/test RMSE (epochs vs rounds).